

Twinkle — building the installer

How to turn the source code into the one file you hand to a clinic.

Handing it over and installing it are a separate job, written up in [INSTALLING.md](#). This document stops when the file exists.

What you need, once

Windows	10 or 11, 64-bit. IExpress, which does the packing, ships with it
.NET SDK 10	<code>dotnet --version</code> should print <code>10.</code> or higher
The repository	Cloned, and it builds — <code>dotnet build HMS_WPF.slnx</code>

Nothing else. No Inno Setup, no WiX, no Visual Studio. The whole thing is one PowerShell script and a tool Windows already has.

The four steps

1. Set the version

Open `src/Pharma.App/Pharma.App.csproj` and change one line:

```
<Version>1.0.1</Version>
```

That single field becomes:

- the version in the navigation sidebar, which support asks people to read out,
- the version listed under **Add or remove programs**,
- the assembly and file version stamped on the executable.

Do this before every release. Two different builds both calling themselves 1.0.0 is the fastest way to spend an afternoon on a bug that was fixed weeks ago. There is nothing in the build that will stop you.

2. Run the tests

```
dotnet test tests\Pharma.Tests\Pharma.Tests.csproj
```

The 240-odd unit tests take about half a minute and cover the pricing, the GST arithmetic, pack sizes and the data health checks — the parts where a mistake costs somebody money.

The UI tests are worth running before a release you are actually shipping:

```
dotnet test tests\Pharma.UiTests\Pharma.UiTests.csproj
```

They drive the real application through the real screens and take six or seven minutes. They open and close windows while they run, so leave the machine alone.

3. Build the installer

```
powershell -ExecutionPolicy Bypass -File scripts\make-installer.ps1
```

Two or three minutes, most of it compressing. It prints its progress:

```
1/3 Building the application as a single file ...  
    72.9 MB  
2/3 Writing the package definition ...  
3/3 Packing (a 73 MB payload takes a minute or two) ...  
  
Setup file: C:\HMS\Setup\TwinkleHMSSetup.exe (66.6 MB)
```

Somewhere other than `C:\HMS\Setup` :

```
powershell -ExecutionPolicy Bypass -File scripts\make-installer.ps1 -OutputDir D:\Handover
```

4. Check what came out

Two quick checks, worth the minute they cost.

The file is about the right size. Roughly 65–70 MB. If it is a few hundred kilobytes, the payload did not go in and you have packaged an installer that installs nothing.

It contains what it should. Unpack it without running it:

```
"C:\HMS\Setup\TwinkleHMSSetup.exe" /C /Q /T:%TEMP%\check
```

`%TEMP%\check` should hold exactly three files — `TwinkleHMS.exe` at about 73 MB, `install.cmd` and `install.ps1` .

Then test it on a machine that is not this one. A clean PC or a virtual machine with no .NET installed and no `C:\HMS` folder. Installing on the machine that built it proves almost nothing: everything it needs is already there. This is the step that catches a missing runtime, and it is the step everybody skips.

What the script actually does

Worth knowing for the day it breaks.

1. **Publishes the application** self-contained, single-file and compressed. Self-contained means the .NET runtime is inside the executable, so the clinic PC needs nothing installed first. That is what makes it 73 MB rather than 5.
2. **Throws away everything else** the publish produced — debug symbols, the Lato fonts QuestPDF ships as package content, `appsettings.json` . None of it is needed beside a single-file build.

3. **Writes an IExpress definition file**, an ini-style `.sed` . IExpress takes no arguments; the file is how you talk to it.
4. **Runs IExpress**, which packs the executable and the two install scripts into one self-extracting file and sets it to run `install.cmd` afterwards.

`scripts/installer/install.ps1` is the part that runs on the clinic PC: it asks for administrator rights once, refuses if Twinkle is open, copies the executable to `C:\HMS\App` , makes the data folders, creates the shortcuts and registers the uninstaller.

When the build fails

What you see	What it is
The string is missing the terminator	A non-ASCII character got into a <code>.ps1</code> . Windows PowerShell reads these as ANSI without a byte order mark, and one em dash in a string stops the file parsing. Keep the scripts plain ASCII
Publish failed	Build the solution on its own to see the real compiler error — the publish output hides it
TwinkleHMS.exe was not produced	The publish succeeded but wrote somewhere else. Check <code>-o</code> against the staging folder in the script
IExpress did not produce ...	Usually <code>TargetName</code> in the <code>.sed</code> pointing at a folder that does not exist. The script creates it, so this means the path is wrong
The file is a few hundred KB	The payload was not picked up. Check that the staging folder still held the executable when IExpress ran
being used by another process	Twinkle is open, or a previous run is still holding the file. Close it

Signing, when it is worth it

The setup file is not code-signed. On the clinic PC that means:

- "Windows protected your PC" on first run — **More info** → **Run anyway**,
- the publisher shown as unknown,
- antivirus occasionally quarantining it, since an unsigned self-extracting executable is a common false positive.

For one or two clinics, warning people that the message is coming is enough. For wider distribution, buy a code-signing certificate and sign both `TwinkleHMS.exe` and `TwinkleHMSSetup.exe` with `signtool` . The warnings stop.

The other two ways to publish

Both still work. Neither produces something you can hand to somebody.

`scripts/publish.ps1` installs straight into `C:\HMS\App` on *this* machine and makes a desktop shortcut. For the development PC.

ClickOnce, from Visual Studio or:

```
msbuild src\Pharma.App\Pharma.App.csproj -t:Publish -p:PublishProfile=ClickOnceProfile -p:Configuration=Release
```

It produces `setup.exe`, `TwinkleHMS.application` and an `Application Files` folder. **All three have to travel together** — the `setup.exe` is only a bootstrapper and fails on its own, which is exactly why it is not what we hand over. Its strength is automatic updates from a shared folder, worth revisiting if the clinic ever runs more than one PC.

Rebuilding this document

```
node scripts\docs-to-pdf\guide_to_pdf.mjs docs\BUILDING_THE_INSTALLER.md docs\BUILDING_THE_INSTALLER.pdf
```